MODULE 3 · AGENTIC RAG · ENGINEERING DEEP-DIVE

Semantic Search RAG vs. *Moment RAG*

Implementing and comparing two retrieval-augmented generation systems over a YouTube transcript — chunk-based retrieval vs. moment-based retrieval.

CALVIN NGUYEN

GITHUB.COM/CALVNGUYEN/SEMANTIC_SEARCH_MOMENT_RAG

01 · PROJECT OVERVIEW

Two systems, *one transcript*, one comparison.

Build a baseline and a moment-aware RAG over the same source, then measure retrieval quality.

Build two RAG systems and *compare* them.

What I built

Baseline Semantic Search RAG — fixed chunks, embeddings, cosine top-k, answer.

Moment RAG — semantic moments, enrichment, hybrid retrieval, decomposition, fusion, re-ranking, cited answers.

How I compare

Same transcript, same models — only the *retrieval design* changes.

Run shared queries through both and judge recall, coverage, grounding, and explainability.

SELECTED VIDEO

“What is a Vector Database?”

Powering Semantic Search & AI Applications.

- Directly related to RAG concepts
- Explains embeddings and vector databases
- Multiple distinct topics — good for semantic segmentation
- Ideal for comparing chunk-based vs. moment-based retrieval

youtube.com/watch?v=gl1r1XV0SLw

One short transcript, measured both ways

109

transcript cues (~1,500 tokens)

`youtube-transcript-api`

9

fixed chunks — baseline

`~256 tok · 25% overlap`

10

semantic moments — Moment RAG

`segment_moments()`

39

hypothetical questions generated

`enrich_moment()`

1536

embedding dimensions

`text-embedding-3-small`

02 · BASELINE SEMANTIC SEARCH RAG

The *conventional* pipeline.

Fixed chunks, one embedding lookup, stuff-and-answer.

Transcript → chunks → embeddings → *answer*

01

Fixed Chunks

~256-token windows, 25% overlap → 9 chunks.

→

02

Embeddings

text-embedding-3-small, 1536-dim vectors.

→

03

Cosine Search

NumPy similarity index → top-k chunks.

→

04

GPT Answer

gpt-4o-mini answers from retrieved context.

Six lines of plumbing.

Build (ingest)

1 · Fetch transcript via *youtube-transcript-api*.

2 · Split into *~256-token chunks*, 25% overlap.

3 · Embed with *text-embedding-3-small* (1536-dim).

Serve (query)

4 · Store vectors in a *NumPy* similarity index.

5 · Retrieve *top-k* by cosine similarity.

6 · Generate the answer with *gpt-4o-mini*.

Where it *struggles*.

Arbitrary structure

Chunk boundaries are set by a *token budget*, not by topic.

A single explanation can be *split across chunks*, so neither half is complete.

One signal

Only *dense cosine* retrieval — no keywords, no intent, no fusion.

One lookup decides everything; multi-part questions get under-answered.

03 · MOMENT RAG

Retrieve *moments*, not arbitrary chunks.

Seven stages: segment, enrich, hybrid retrieve, decompose, fuse, re-rank, ground.

Build the moment index *once*

01

Cues

109 timed transcript cues.

→

02

Segment

Group at semantic breakpoints → 10 moments.

→

03

Enrich

GPT: gist, keywords, 39 hypothetical questions.

→

04

Index

Text + question embeddings + BM25.

Intelligence on the *query* side

01

Decompose

One question → 2–4 sub-queries.

→

02

Hybrid Retrieve

Dense text + dense questions + BM25.

→

03

Fuse & Rerank

RRF fusion, then cross-encoder re-rank.

→

04

Ground

Cited answer with timestamp deep-links.

STEP 1

Segment by *meaning*

- Embed every transcript cue
- Measure embedding distance between adjacent cues
- New moment when distance exceeds a threshold
- Output: **10 semantic moments**

MOMENT
2:43 → 3:24

TOPIC
How unstructured data becomes embeddings

STEP 2

For each moment, GPT writes…

- *Gist* — one-sentence summary
- *Keywords* — important concepts
- *Hypothetical questions* — what it answers

Purpose: improve recall and query-to-moment matching.

MOMENT TEXT

"An embedding is an array of numbers…"

GENERATED QUESTIONS

What is an embedding?
How are embeddings created?
Why are embeddings useful?

STEP 3

Three retrieval signals

Each captures something the others miss.

- *Dense text* — query vs. moment-text embeddings → captures **meaning**
- *Dense questions* — query vs. generated-question embeddings → captures **intent**
- *BM25* — keyword matching → captures **exact terms & acronyms**

Split multi-part questions *before* retrieval.

User query

“How does a vector database index embeddings and search them quickly?”

Purpose: cover every facet of a multi-part question instead of just one.

Generated sub-queries

→ What is a vector database?

→ How are embeddings indexed?

→ What techniques make search fast?

Combine the signals, then *sharpen* them.

Step 5 · RRF Fusion

Reciprocal Rank Fusion merges the dense, question, and BM25 rankings into one.

Moments that rank highly across *multiple methods* get the highest final score.

Step 6 · Cross-Encoder Re-rank

Model: *cross-encoder/ms-marco-MiniLM-L-6-v2*.

Re-scores candidates against the original query. Embeddings are fast but approximate; the cross-encoder is *slower but more accurate*.

STEP 7

Answers you can *verify*

- GPT-generated answer
- Timestamp citations
- Direct YouTube deep links

Users verify each claim straight from the source video.

CITATION
[7:53]

DEEP LINK
youtube.com/watch?v=gl1r1XVOSLw&t=473s

04 · RESULTS & COMPARISON

Same transcript. *Different* answers.

The gains come from retrieval design, not a bigger model.

BASELINE — FAILED

Incomplete explanation

“The context does not explicitly define what an embedding is.”

— Definition split across fixed chunks

MOMENT RAG — ANSWERED

Correct, cited answer

“An array of numbers that captures the semantic essence of data — similar items sit close together in vector space.”

— Retrieved the right moment + timestamp citations

BASELINE

- Fixed chunks
- Dense retrieval only
- Single query
- Cosine similarity ranking

MOMENT RAG

- Semantic moments
- Dense + Question + BM25 retrieval
- Query decomposition
- RRF fusion + cross-encoder re-ranking
- Timestamp citations

KEY FINDINGS

Moment RAG improved…

Same source, same model — better retrieval design.

- *Recall* — finds the relevant moment more reliably
- *Multi-part coverage* — decomposition answers every facet
- *Retrieval quality* — hybrid + re-rank sharpen results
- *Grounding* — timestamp citations into the video
- *Explainability* — you can see why each moment was chosen

Limitations & *future work*.

Limitations

Small transcript dataset.

GPT enrichment cost (one call per moment).

Additional retrieval latency.

Future improvements

Replace NumPy search with *ChromaDB or Qdrant*.

Larger evaluation dataset + automated metrics: *Recall@K, Precision@K, MRR*.

Better semantic segmentation.

MODULE 3 · AGENTIC RAG

Better retrieval, *not* a bigger model.

Calvin Nguyen ·
github.com/calvnguyen/semantic_search_moment_rag